

Ricerca sulle tecniche di offuscamento dell'Hardware

Stato dell'arte: Attacchi e Difese

Dimitri Binosi

Università degli Studi di Verona

Anno Accademico 2025-2026

- **Il problema:** Mercato dei componenti contraffatti e perdite economiche ingenti.
- **Rischi:** Sovraproduzione non autorizzata, pirateria IP e clonazione fisica.
- **Sicurezza Critica:** Nel settore automotive, vulnerabilità hardware possono causare danni fisici.
- **La Soluzione:** L'offuscamento dell'hardware permettere l'utilizzo solo ad utenti autorizzati tramite chiavi segrete.

Cos'è l'Offuscamento dell'Hardware?

Si divide principalmente in due categorie:

Offuscamento del Core DSP (Strutturale)

Trasformazione del Data Flow Graph (DFG) per rendere oscura la struttura a livello RTL o gate.

Offuscamento Combinatorio (Logic Locking)

Modifica del circuito tramite l'inserimento di porte logiche (**key-gates**) come MUX, XOR o LUT. Il circuito funziona correttamente solo se viene fornita la chiave esatta.

L'aggiunta della programmabilità avviene tipicamente tra la sintesi logica e quella fisica.

- **XOR/XNOR:** Inverte il segnale se il bit di chiave è errato.
- **MUX:** Seleziona tra il segnale corretto e uno errato/falso.
- **LUT:** Sostituisce porte standard con tabelle programmabili.

Assunzioni:

- L'attaccante ha accesso alla *Netlist* (schema del circuito bloccato).
- L'attaccante ha un *Oracle* (chip sbloccato) per osservare le uscite.

Strategie e Complessità:

- **Brute Force:** Inefficace. Complessità esponenziale.
- **ATPG:** Sfrutta pattern di test, ma fallisce se i pattern non sono significativi.
- **SAT Solver base:** a causa della natura del problema SAT risulta inefficace.

Proposto inizialmente da Subramanyan et al. (2015), è un attacco di tipo **Oracle-guided**.

- **Funzionamento:** Utilizza i *Distinguishing Input Patterns* (DIP) per eliminare gruppi di chiavi errate.
- **Efficienza:** Ad ogni iterazione, il SAT solver trova un input che produce output diversi per due chiavi ipotetiche.
- **Obiettivo:** Restringere lo spazio delle chiavi fino a trovare quella corretta (o una equivalente).

Algoritmo SAT-Attack

Require: Circuito bloccato C , Circuito sbloccato $eval$

```
1:  $F_1 \leftarrow C(X, K_1, Y_1) \wedge C(X, K_2, Y_2)$ 
2: while  $sat(F_i \wedge (Y_1 \neq Y_2))$  do
3:    $X_d \leftarrow sat\_assignment(F_i \wedge (Y_1 \neq Y_2))$ 
4:    $Y_d \leftarrow eval(X_d)$ 
5:    $F_{i+1} \leftarrow F_i \wedge C(X_d, K_1, Y_d) \wedge C(X_d, K_2, Y_d)$ 
6: end while
7: return  $K_C \leftarrow sat\_assignment_{K_1}(F_i)$ 
```

Non tutti gli attacchi si basano sulla logica booleana pura.

- **Removal Attack:** Identifica e rimuove fisicamente i blocchi di locking se questi sono strutturalmente isolabili o ridondanti.
- **Valkyrie:** Un tool diagnostico che analizza la vulnerabilità strutturale della netlist.
- **DIP critici:** Esistono i *remove-all DIP*, pattern capaci di rivelare la chiave istantaneamente.

Difese contro gli Attacchi SAT

Le difese moderne si dividono in due filosofie:

Approccio	Meccanismo
SAT-Hard Blocks	Aumentano la complessità computazionale di ogni iterazione del solver.
Point Functions	Inseriscono funzioni (Anti-SAT) che richiedono un numero esponenziale di iterazioni.

Criticità: Bilanciare la resistenza con l'overhead di area (superficie del chip).

Tecnica Two-Level-Based

La difesa **Two-Level-Based** (Anti-SAT) affronta SAT, Removal e Valkyrie simultaneamente.

- **Split-Compensate:** Espande l'AND-tree introducendo variabili esterne (area) e interne (resistenza ai DIP).
- **Flusso Operativo:** Localizza l'AND-tree → Verifica i vincoli → Cifra → Ottimizza il cono logico.

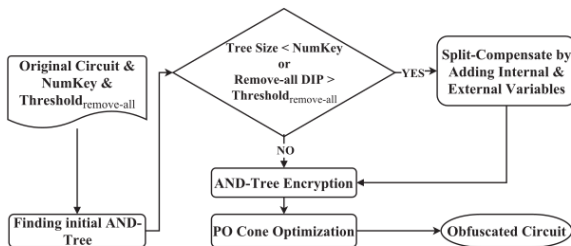


Figura: Flusso della tecnica two-level-based

Conclusione

L'offuscamento è una sfida continua tra design e attacchi. Il futuro risiede nel bilanciare la sicurezza matematica con l'efficienza strutturale.